PanaceaApp

DEFINE · DESIGN · DEVELOP · DEPLOY

AI-ENABLED SOFTWARE DELIVERY

From nothing to a shipped application.

Most tools put AI inside one box of the lifecycle. PanaceaApp owns the whole journey: a fleet of coding agents on isolated machines, and a single provenance thread that ties the thing you shipped back to the thing you asked for.

DOCUMENT **PA-01-AI-SDLC** VERSION **1.0** DATE **28 SEPTEMBER 2026** OWNER **KPMG AUSTRALIA**

CONFIDENTIAL, KPMG AND AUTHORISED RECIPIENTS

SECTION ONE

What an AI-enabled SDLC actually is

The phrase is used loosely. Most of what it describes is a faster autocomplete inside an unchanged lifecycle. Three things have to change before the lifecycle itself is different, and all three are structural rather than cosmetic.

1 The category error

Today's tooling drops an assistant into one box of the lifecycle, almost always the box marked *write code*. Everything around it is untouched. A person still carries the requirement into the design by reading it and retyping it, carries the design into the build the same way, and reconstructs from memory, at release time, why any of it was built. The assistant is faster. The lifecycle is identical, and every handoff is still a human copying context between tools that do not know about each other.

That is worth something, and it is not a new SDLC. It is the old one with a better editor.

2 The three shifts

Work becomes parallel
Not one conversation at a time. A fleet of agents working simultaneously, each in its own isolated machine and its own branch, with one place to watch them all.

Context becomes an artifact
The brief is a versioned object with an identity, not a paragraph someone pasted. It is referenced, not retyped, and it survives every handoff.

Done becomes proven
Each acceptance criterion carries its own evidence. A criterion with no proof attached is not complete, whatever anybody predicts about it.

Define → Design → Develop → Deploy

FOUR STATES OF ONE PROJECT, JOINED BY ONE THREAD

3 Why the whole journey, and not a better box

The value is not in any single phase. It is in the seam between phases, which is exactly where software delivery loses fidelity today and exactly where nobody has put anything. A platform that owns Define through Deploy can make a guarantee no point tool can make: that the release you are about to sign off can be traced, mechanically and without anyone's recollection, to the requirement that asked for it.

THE BAR THIS PRODUCT IS HELD TO

Usability and iteration on powerful tools, not raw capability. A feature that is powerful but takes a wall of configuration to reach has failed the bar and does not ship.

SECTION TWO

One project, four states

The four phases are not four tools with integrations between them. They are four states of the same project, and the same artifact is visible in all of them.

PHASE	WHAT THE PERSON DOES	WHAT THE AGENTS DO	WHAT THE THREAD CARRIES
Define	States the problem and the acceptance criteria, in their own words.	Interrogate the brief, surface what is missing, propose criteria that can be proven.	An immutable brief revision with its own key and digest.
Design	Judges what it should look and feel like, on a real canvas.	Edit real frames through a tool surface, not by describing changes in prose.	Every frame references the brief revision it serves.
Develop	Sets direction, reviews, and decides. Watches a fleet rather than a cursor.	Work in parallel, one isolated machine and one branch each, converging back.	Each branch carries the brief revision as context that agents cannot silently drop.
Deploy	Approves a release against evidence rather than a summary.	Assemble the release, attach the proof for each criterion, and ship it.	The release is pinned to the exact brief revision it satisfies.

4 The shape of a session

1. **A brief is written and frozen**
   It gets an identity and a digest. Later edits create a new revision; they never mutate the one already referenced.
2. **Machines come up**
   One isolated microVM per session, warm, with the project already checked out into its own working tree.
3. **Agents work in parallel**
   Each in its own branch. Their terminals stream into one cockpit, so the person supervises rather than waits.
4. **Work converges**
   Branches merge back explicitly, with a compare and swap on the reference so two agents can never quietly overwrite one another, and with an undo.
5. **The release carries its proof**
   Each criterion from Define arrives at Deploy with the evidence that discharges it, or it is visibly not done.

SECTION THREE

The provenance thread

This is the spine of the product and the reason the four phases are worth owning together. It is also the part that matters most to a regulated customer, because it converts "we believe this is why we built it" into something a machine can check.

5 An identity, not a paragraph

A brief revision is an immutable object with a key and a content digest. Nothing downstream copies its text. Design frames, development branches and releases all reference it, so a change to the requirement produces a new revision and leaves a visible gap wherever the old one is still being relied on. The thread is therefore not documentation that goes stale. It is a structure that breaks loudly when reality moves.

`brief:<id>@<rev>`

REFERENCED IN DESIGN, CARRIED INTO DEVELOP, ATTACHED AT DEPLOY

6 Done means proven

Every acceptance criterion written in Define must arrive at Deploy with the evidence that discharges it. Not a prediction that a test will pass, and not a summary of what was attempted: the artifact itself, attached to the criterion, produced by a run that actually happened. A criterion with nothing attached is reported as not done, which is a far more useful answer than an optimistic one.

What this buys an engineer

No archaeology. The answer to "why does this code exist" is a link, not an afternoon in a commit log.

What this buys an auditor

Traceability from requirement to release that was produced as a by-product of doing the work, rather than assembled afterwards for the audit.

THE RULE THIS CREATES

Any feature that breaks the thread is wrong, regardless of how good it is in isolation. That constraint has already removed capable designs from this product, and it is the constraint that makes the rest of it worth having.

SECTION FOUR

A fleet, not a chat window

The unit of work is a session on its own machine, and a person runs many at once. The cockpit exists so that supervising ten of them costs less attention than driving one.

1	1	4	0
MICROVM PER SESSION	WORKING TREE PER AGENT	PHASES, ONE PROJECT	DATA RETAINED BY INFERENCE

7 Isolation that is real

Each session runs in its own microVM with its own kernel, not a shared container runtime. An agent is treated as untrusted code, because that is what it is: it writes files, runs commands and follows instructions from text it was given. The host never executes anything the sandbox can write, and no credential the customer cares about is ever placed inside one.

Warm pools keep this fast. Isolation of this strength costs seconds to start cold, so machines are made ready before anybody asks for one.

8 Parallel work that converges

Every agent gets its own working tree, so two agents cannot corrupt each other's state by sharing a checkout. Work returns along explicit lanes, and a lane lands with a compare and swap on the git reference, so a merge cannot silently discard a peer's result. Anything that lands can be undone.

9 Teams, without agents talking past each other

A project shared with a team shows each member's work, agents and focus. Agents are aware of what other agents are doing as structured facts rather than as free text, and each person has one orchestrator that negotiates with other people's orchestrators. Nobody's agent takes instructions from somebody else's, which keeps a collaboration channel from becoming an injection channel.

REAL PTYS OVER WEBSOCKET · UNMODIFIED AGENT CLIS · ONE HEARTBEAT, NO SPINNERS · LOCAL MACHINE OR CLOUD MACHINE

SECTION FIVE

Built for a regulated enterprise

The wedge is not raw capability. It is that this can be run by an organisation that has to answer for where its code, its data and its credentials went.

10 Where the work happens

ISOLATION	One microVM per session, with its own kernel, on pod sandboxing. Read only root, no host networking, explicit CPU and memory.
HOSTING	The customer's own cloud tenancy. The platform is delivered into their environment rather than their source being delivered into ours.
IDENTITY	The person signs in as themselves, through the organisation's existing directory, with no new registration to raise and nothing to click.
CREDENTIALS	Tokens are held by the operating system's credential store. A sandbox receives a scoped, short lived, per session token and never an organisation key.
AUDIT	Append only. Brief revisions and the ledger are immutable by construction, not by convention.

11 Where the data goes

Inference runs through an organisation level gateway with zero data retention. No personal subscriptions are used and no organisation key reaches a sandbox. Each project carries a data classification set when it is created, and that classification decides the route: an Australian data project is served in country, and a project that may not leave the country fails closed rather than falling back.

> **FAIL CLOSED, NOT FAIL QUIET**
>
> A residency guarantee is only worth the test that proves it. The route is asserted on what actually leaves the network, not on what the configuration says it should do, because a gateway that ignores its own configuration on one endpoint is a real and observed failure mode.

12 What the platform is made of

Third party components are permitted only under permissive licences. Copyleft and source-available licences are excluded from the shipped product, and every dependency's licence is verified from its own licence file rather than from a badge. This is enforced because the product is delivered into customer tenancies, where a licence defect found late is a rewrite rather than a conversation.

SECTION SIX

Where it is today

This programme reports what it has proven and what it has not. The distinction is the point of the product, so the document that describes the product should hold to it.

CAPABILITY	STATE	EVIDENCE
A real coding agent driven end to end through an isolated sandbox	PROVEN	A live session driven to completion against a container backend, with the terminal, resize, colour and exit status all carried faithfully.
The cockpit: many live sessions, one window	PARTLY PROVEN	A nineteen check end to end harness run in full against a real control plane, real containers and the native window. Ten checks passed. Of the nine that did not, five were a defect in the test itself, three were one product defect since fixed, and one was a timeout since fixed. The run has not yet been repeated against those fixes.
Sessions survive detach, reload and a control plane restart	PROVEN	Twelve live sessions re-attached after a cockpit reload; sessions and their containers survived a control plane restart unaided.
Git inside a sandbox on the corporate network	PROVEN	Work committed from inside a sandbox as the configured identity, through the intercepting proxy, with the certificate bundle mounted.
Git lanes, worktrees and convergence	IN BUILD	Planned and under construction, with the security ruling that host git never runs anywhere a sandbox can write.
The Hub: briefs, approvals, ledger, releases	IN BUILD	Identity and schema landed; the ledger and release surfaces are in progress.
Define, Design and Deploy surfaces	IN BUILD	Planned in full, with adversarial plan review complete for each.
Cloud isolation on managed Kubernetes	PLANNED	Written and reviewed, deliberately never applied. It needs a real cluster, and nothing claims it works until it runs on one.

NOT YET PROVEN, AND STATED PLAINLY

MicroVM isolation, cold start and the read only root have not been exercised on a real cluster. The visual acceptance of the cockpit is the product owner's to give and has not been given. Neither is claimed anywhere in this document.

SECTION SEVEN

The product is built the way it asks you to build

PanaceaApp is being developed by the method it sells: agents working in parallel under review gates, with evidence attached to every claim. That is the strongest available argument that the method works, and it is also where its sharp edges were found.

13 Every task passes a gate

A task is planned, then implemented, then reviewed by an independent agent that re-runs the proofs itself rather than reading the report. The reviewer is instructed to attack the work: to name the mutations it expects to survive the test suite and run them, and to check whether any assertion was quietly weakened to make something pass. A task is complete when the evidence survives that, not when the author says so.

14 What that catches, with one example

An end to end run reported five failures that looked like a deep fault in how the product reconnects to many concurrent sessions. An adversarial investigation found something else: a single test scrolled the terminal using a gesture that moves a fixed five rows regardless of the distance requested, a fact the same test file had measured and written down a hundred and seventy lines earlier. That one failure then left a background loop running in a shared shell, and the four failures after it were its wake.

The product was not at fault, and without the investigation the next weeks would have gone into hardening a component that was never broken. A test that is wrong is more expensive than a test that fails.

THE STANDING RULE

Never re-run a red result until it goes green, and never loosen a bound to make something pass. Every false pass this programme has caught is written down in the verification record with what produced it, because each one would otherwise have shipped as proof of something that had never been observed.

15 Where this goes next

Near term
Close the cockpit, land it on the main line, and put it in the product owner's hands to use for real work.

Then
Git convergence and the Hub, which together turn a fleet of terminals into a delivery system with a memory.

After that
Define, Design and Deploy, closing the thread end to end, and the cloud environment that carries it at scale.

Throughout
Team mode and the work layer, so that a project is something a group runs together rather than a person runs alone.